

Abbreviated Lambdas for Fun and Profit

Contents

- [Motivation](#)
- [Proposal](#)
 - [=> expr](#)
 - [Unary operator>>](#)
 - [Omitting type names](#)
- [Examples](#)
- [Effects on Existing Code](#)
- [Prior Work](#)
- [Acknowledgements and References](#)

Motivation

There are two, somewhat related motivations for an abbreviated lambda syntax. The first is to address the problem of trying to pass in overload sets as function arguments [\[1\]](#):

```
template <class T>
T twice(T x) { return x + x; }

template <class I>
void f(I first, I last) {
    transform(first, last, twice); // error
}
```

C++14 generic lambdas allow us a way to solve this problem by just wrapping the overloaded name in a lambda:

```
transform(first, last, [](auto&& x) {
    return twice(std::forward<decltype(x)>(x));
});
```

But that isn't actually correct, although it's the "obvious" code that most people would produce, and the problems are pretty subtle. The first problem is that it fails to return a reference where the function returned a reference, so there's an extra copy here in some cases (or just a hard error, if the type happens to be noncopyable). That's an easy fix though, just add the appropriate trailing return type:

```
transform(first, last, [](auto&& x) -> decltype(auto) {
    return twice(std::forward<decltype(x)>(x));
});
```

Now we always have the right return type. But it's still not quite right: there's no condition checking on the argument, so the lambda isn't SFINAE-friendly. This could lead to hard errors where it's used in a situation where that matters, such as:

```
struct Widget;

bool test(int );
bool test(Widget );

void invoke(std::function<bool(int)> ); // #1
void invoke(std::function<bool(std::string)> ); // #2

// error: unresolved overloaded function type
invoke(test);

// error with a really long stack trace
invoke([](auto&& x) -> decltype(auto) {
    return test(std::forward<decltype(x)>(x));
});
```

Situations like this come up from time to time, and will probably only become more common with the introduction of `std::variant` and the ability to easily use lambdas in `std::visit()` thanks to variadic using declarations [\[2\]](#).

The real, correct, way to lift an overloaded function into a function object would involve writing all of this (and even *this* version isn't `noexcept`-correct, which may matter in some cases!):

```
// finally OK: calls #1
invoke([](auto&& x)
    -> decltype(test(std::forward<decltype(x)>(x)))
{
    return test(std::forward<decltype(x)>(x));
});
```

That's a **lot** to have to type for what's conceptually a very simple thing: passing a function, that happens to be overloaded, to a function template. That lambda in of itself is 108 characters long, most of which is boilerplate that drowns out the important parts of the call (which I highlighted in **blue**). That's far too long. Moreover, it's literally too long for many slides, so what you see at talks at conferences is typically people omit the perfect-forwarding for brevity's sake, and the forwarding syntax itself is so long that people just use macros:

```
#define FWD(x) static_cast<decltype(x)&&>(x)

[](auto&& x) -> decltype(test(FWD(x))) { return test(FWD(x)); }
```

Now we're down to just 64 characters. This is a good deal better, and we can even wrap the entire lifting process into a lambda to make it even shorter, but do we really want to be in a position where we're recommending the use of macros?

Another place where this `FWD` macro comes especially handy is when we want to forward arguments *into* a lambda, just for normal purposes:

```
template <class T>
void foo(T&& arg) {
    bar([arg=std::forward<T>(arg)]{ ... });
}
```

Again, that's so much to type. If I was copying `arg`, I could just type `arg` or `&arg`. If I was taking `arg` by reference, I could write `&arg` or `&`. But with forwarding, which is a fairly common operation, I need to write this 28-character beast (or, with macro, 13). But at least I *can* forward a single argument verbosely. With a parameter pack, I can capture the pack by copy or by reference, but to capture the whole thing by forward I would need to put it into a `std::tuple`. That seems very unsatisfying.

These all seem like major annoyances in modern C++. Too much code to write simple things.

Proposal

This paper proposes three language extensions. The extensions themselves are independent, but together combine to allow for terse, readable, correct lambdas and functions.

=> expr

This paper proposes the creation of a new lambda introducer, `=>`, which allows for a single expression in the body that will be its return statement. This will synthesize a SFINAE-friendly, `noexcept`-correct lambda by doing the code triplication for you.

That is, the lambda:

```
[](auto&& x) => test(x)
```

shall be exactly equivalent to the lambda:

```
[](auto&& x) noexcept(noexcept(test(x))) -> decltype(test(x)) { return test(x); }
```

When SFINAE is important, the repetition of the function body makes the code difficult to read. At best. At worst, the code becomes error-prone when one changes the body of the function while forgetting to change the body of the trailing `decltype`. Even when SFINAE is not important, for the simplest lambdas, brevity is important for readability and omitting the `return` keyword would be helpful.

The code duplication or triplication in short function bodies is also a common problem that wants for a solution [3], but this proposal is focused solely on lambda expressions.

Unary operator>>

Another source of boilerplate is generic code is `std::forward`. When dealing with templates, the uses of `forward` can overwhelm all the rest of the code, to the point where many talks and examples just omit references entirely to save space and many programmers just give up trying to read it. Hence the viability of the `FWD()` macro presented earlier.

Unlike `std::move` and `std::ref`, which are used to do non-typical things and deserve to be visible markers for code readability, `std::forward` is very typically used in the context of using forwarding references. It does not have as clear a need to be a signpost.

This paper would like to see a shorter way to forward arguments and proposes non-overloadable unary operator `>>`, where `>>expr` shall be defined as `static_cast<decltype(expr)&&>(expr)`, and shall not overloadable. Furthermore, the symbol `>>` in the capture of a lambda expression can be used to "decay-copy" [4] the named variable or all variables.

This lambda:

```
[](auto&& x) { return test(>>x); }
```

is equivalent by definition to this one:

```
[](auto&& x) { return test(std::forward<decltype(x)>(x)); }
```

This function (a one-time delayed invoker):

```
template <class F, class... Args>
auto delay_invoke(F&& f, Args&&... args) {
    return [>>]() => std::invoke(>>f, >>args...);
}
```

is logically, though not strictly, equivalent to this one:

```
template <class F, class... Args>
auto delay_invoke(F&& f, Args&&... args) {
    return [f=std::forward<F>(f), tup=std::forward_as_tuple(std::forward<Args>(args)...)]() -> decltype(auto) {
        return std::apply(std::forward<F>(f), std::move(tup));
    };
}
```

Using `>>` to forward doesn't just make the code shorter. It makes it easier to read and easier to write. In the second example, it let's us naturally and directly invoke the function without having to do the indirection through a tuple (which is is neither easy to read nor write nor reason about). It's also probably easier for the compiler too.

Unary operator`>>` will have equivalent precedence to the other prefix operators, like `operator!`.

Omission of type names in lambdas

One of the motivations of generic lambdas was to use `auto` as a substitute for long type names, which helps quite a bit. But since `auto&&` has become such a regular choice of argument for lambdas, and is rarely a wrong choice, it doesn't really bestow any information to the code. It would be great if we could allow the type name to be omitted, in which case it will be assumed to be `auto&&`. That is, if:

```
[](x) { return x+1; }
```

could be exactly equivalent to the lambda:

```
[](auto&& x) { return x+1; }
```

However, this is impossible to do for all lambdas. The example:

```
struct arg { arg(int ) { } };
auto lambda = [](arg ) { return sizeof(arg); }
int x = lambda(1);
```

is well-formed today, yielding a value of (typically) 1 for `x`, and we can't just change that to yield (typically) 4. But while there's little we can do here, we can do something for precisely those simple lambdas that this proposal is focused on. The fragment `[](x) => expr` currently is ill-formed. So this paper proposes, for those abbreviated lambdas using the `=>` syntax, to allow for the omission of type names. Any standalone identifier shall be a parameter of type `auto&&`. Omitted type names can be interspersed with provided type names, and a trailing `...` will indicate a parameter pack of forwarding references. That is, the lambdas:

```
[](x, int y) => x < y
[](args...) => test(>>args...)
```

shall be exactly equivalent to the C++14 lambdas

```
[](auto&& x, int y) noexcept(...) -> decltype(x < y) { return x < y; }
[](auto&&... args) noexcept(...) -> decltype(test(std::forward<decltype(args)>(args)...)) { return test(std::forward<decltype(args)>(args)...); }
```

No default arguments will be allowed in the case of type omission, due to potential ambiguities in parsing.

Alternative methods to omit type names

The goal of this proposal is to ultimately allow for the omission of type names in lambdas, defaulting to `auto&&`. The specific mechanism proposed for achieving this goal (simplying allowing omission the case of `=>` lambdas, requiring lookahead parsing) is just one such possible way of solving this problem. Other alternatives include:

- Using a placeholder (e.g. `:`) to indicate that all type will be omitted. It would be impossible to specify an unnamed argument of a specific type.

```
[]@(x, y) // equivalent to [](auto&& x, auto&& y)
[]@(args...) // equivalent to [](auto&&... args)
[]@(int ) // ill-formed
```

- Using a placeholder (e.g. `:` or `^`) for a particular argument to indicate that its type name is omitted. With each parameter individually marked, it would be possible to specify an unnamed argument

```
[](@x, @y) // equivalent to [](auto&& x, auto&& y)
[](@args...) // equivalent to [](auto&&... args)
[](@x, y) // equivalent to [](auto&& x, y <unnamed> )
```

- As in the original generic lambda proposal [5], allow for the omission of the *type-specifier*:

```
[](&&x, &&y) // equivalent to [](auto&& x, auto&& y)
[](const&... args) // equivalent to [](auto const&... args)
```

Each of these alternatives uses additional indicators to allow for the omission of type names, and so would be just as valid in current lambdas as in those using the proposed `=>` syntax.

Examples

Putting all three features together, binding an overload member function, `func`, to an instance, `obj`, as a function argument is reduced from:

```
[&obj](auto&&... args) noexcept(noexcept(obj.func(std::forward<decltype(args)>(args)...))) -> decltype(obj.func(std::forward<decltype(args)>(args)...)) { return obj.func(std::forward<decltype(args)>(args)...); }
```

to

```
[&obj](args...) => obj.func(>>args...)
```

That is a reduction from 212 (144 if you don't care about noexcept) characters to 39.

Here are other examples of improved usage as compared to C++14 best practices.

Sorting in decreasing order: same typing, but arguably clearer:

```
std::sort(begin(v), end(v), [](auto&& x, auto&& y) { return x > y; }); // C++14
std::sort(begin(v), end(v), std::greater<>{}); // C++14 with function object
std::sort(begin(v), end(v), [](x,y) => x > y); // this proposal - don't need to know about std::greater
```

Sorting in decreasing order by ID:

```
std::sort(begin(v), end(v), [](auto&& x, auto&& y) { return x.id > y.id; }); // C++14
std::sort(begin(v), end(v), std::greater<>{}, &Object::id); // ranges with projections
std::sort(begin(v), end(v), [](x,y) => x.id > y.id); // this proposal
```

Finding an element based on a member function call, with and without additional arguments:

```
auto it = std::find_if(begin(v), end(v), [](auto&& e) { return e.isMatch(); }); // C++14 with lambda
auto it = std::find_if(begin(v), end(v), std::mem_fn(&Object::isMatch)); // C++11 with mem_fn
auto it = std::find_if(begin(v), end(v), [](e) => e.isMatch()); // this proposal

auto it = std::find_if(begin(v), end(v), std::bind(&Object::matches, _1, std::ref(target)));
auto it = std::find_if(begin(v), end(v), [&](auto&& e) { return e.matches(target); });
auto it = std::find_if(begin(v), end(v), [&](e) => e.matches(target));
```

Calling an overload where SFINAE matters and getting it wrong is a mess:

```
bool invoke(std::function<bool(int)> f); // #1
bool invoke(std::function<bool(std::string)> f); // #2

invoke([](auto x) { return x == 2; }); // error! (283 lines of diagnostic on gcc)
invoke([](auto x) -> decltype(x == 2) { return x == 2; }); // OK C++14: calls #1
invoke([](x) => x == 2); // OK this proposal: calls #1
```

Implementing currying, in C++17. This is dramatically more concise than what you would otherwise have to do in C++17 [6], thanks to being able to capture parameter packs by decay-copy and easily get the SFINAE right (here it is critical) without duplication.

```
template <class F>
auto curry(F&& f) {
    return [>](auto&&... args) -> decltype(auto) {
        if constexpr (std::is_callable<F&(decltype(args)...>>::value) {
            return std::invoke(f, >>args...);
        }
        else {
            return curry([>, f](auto&&... new_args) =>
                std::invoke(f, args..., >>new_args...
            );
        }
    };
}
```

Effects on Existing Code

The token => can appear in code in rare cases, such as in the context of passing a the address of the assignment operator as a template non-template parameter, as in X<Y::operator= >. However, such usage is incredibly rare, so this proposal would have very limited effect on existing code. Thanks to Richard Smith for doing a search.

Unary operator>> cannot appear in legal code today, or in this proposed context in a lambda-capture, so that is a pure language extension.

Prior Work

The original paper introducing what are now generic lambdas [5] also proposed extensions for omitting the type-specifier and dropping the body of a lambda if it's a single expression. This paper provides a different path towards those that same goal.

The usage of => (or the similar ->) in the context of lambdas appears in many, many programming languages of all varieties. A non-exhaustive sampling: C#, D, Erlang, F#, Haskell, Java, JavaScript, ML, OCaml, Swift. The widespread use is strongly suggestive that the syntax is easy to read and quite useful.

There has also been an idea floating around, if not a specific proposal, to lift an overloaded function into a lambda with the syntax []f. This is certainly far more concise than what is proposed as an alternative in this paper ([](args...) => f(>>args...)).

Acknowledgements and References

Thanks to Andrew Sutton and Tomasz Kaminski for considering and rejecting several bad iterations of this proposal. Thanks to Richard Smith for looking into the practicality of this design. Thanks to Nicol Bolas for refocusing the paper as three independent language extensions. Thanks to John Shaw for putting up with a torrent of bad ideas.

[1] [Overload sets as function arguments](#)

[2] [Pack expansions in using-declarations](#)

[3] [Return type deduction and SFINAE](#)

[4] [thread.decaycopy](#) defines this as

```
template <class T>
decay_t<T> decay_copy(T&& v) {
    return std::forward<T>(v);
}
```

[5] [Proposal for Generic \(Polymorphic\) Lambda Expressions](#)

[6] [zero-overhead C++17 currying & partial application](#), most of the implementation revolves around properly forward-capturing the parameter packs